1. Keep unit interfaces small (Parametroak maximo 4-ra limitatu)

1. Hasierako kodea:

```
  public Sale createSale(String title, String description, int
status, float price, Date pubDate, String sellerEmail, File
file)
            throws FileNotUploadedException,
MustBeLaterThanTodayException, SaleAlreadyExistException {

        if (pubDate.before(UtilDate.trim(new Date()))) {
            throw new
MustBeLaterThanTodayException(ResourceBundle.getBundle("Etique
tas").getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
        }
        if (file == null) throw new
FileNotUploadedException("No file");

        db.getTransaction().begin();
        try {//Erroreak sahiesteko
            Registered seller = db.find(Registered.class,
sellerEmail);
            if (seller == null) {
                db.getTransaction().rollback();
                return null;
            }

            if (seller.doesSaleExist(title)) {
                db.getTransaction().rollback();
                throw new SaleAlreadyExistException("Sale
already exists");
            }

            Sale sale = seller.addSale(title, description,
status, price, pubDate, file, false);
            db.persist(sale);
            db.getTransaction().commit();
            return sale;
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
            throw e;
        }
```

```
}
```

2. Errefaktorizatutako kodea:

```
public static class SaleRequest {
        public String title;
        public String description;
        public int status;
        public float price;
        public Date pubDate;
        public File file;
        public SaleRequest(String title, String description,
                        int status, float price, Date pubDate, File file) {
                this.title = title;
                this.description = description;
                this.status = status;
                this.price = price;
                this.pubDate = pubDate;
                this.file = file;
        }
}

public Sale createSale(SaleRequest request, String
                sellerEmail)
                                throws FileNotUploadedException,
                                MustBeLaterThanTodayException, SaleAlreadyExistException {
        if (request.pubDate.before(UtilDate.trim(new Date())))
        {
                throw new

MustBeLaterThanTodayException(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.Err
orSaleMustBeLaterThanToday"));
        }
        if (request.file == null) throw new
        FileNotUploadedException("No file");
        db.getTransaction().begin();
        try {
                Registered seller = db.find(Registered.class,
                                sellerEmail);
                if (seller == null) {
                        db.getTransaction().rollback();
                        return null;
                }
                if (seller.doesSaleExist(request.title)) {
                        db.getTransaction().rollback();
                        throw new SaleAlreadyExistException("Salealready exists");
                }
                Sale sale = seller.addSale(request.title,
                                request.description, request.status, request.price,
                                request.pubDate, request.file, false);
```

```
                db.persist(sale);
                db.getTransaction().commit();
                return sale;
        } catch (Exception e) {
                if (db.getTransaction().isActive())
                        db.getTransaction().rollback();
                throw e;
        }
}
```

3. Deskribapena: Produktuaren datuak biltzeko SaleRequest klasea (Parameter Object) sortu da. Horrela, parametro kopurua 7tik 2ra murriztu da irakurgarritasuna hobetzeko.

4. Egilea: Markel Sanchez

2. Duplicate code (Kode errepikatua ekidin)

1. Hasierako kodea:

```
  public void addMoney(String email, float amount) {
        db.getTransaction().begin();
        try {
            Registered u = db.find(Registered.class, email);
            if (u != null) {
                float kop = u.getMoneyKop();
                u.setMoneyKop(kop + amount);
                String deskripzioa =
ResourceBundle.getBundle("Etiquetas").getString("MoneyGUI.jBut
tonAdd.text");
                addMovement(email, deskripzioa, amount);
            }
            db.getTransaction().commit();
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
        }
    }

    public void quitMoney(String email, float amount) {
        db.getTransaction().begin();
        try {
            Registered u = db.find(Registered.class, email);
            if (u != null) {
                float kop = u.getMoneyKop();
                u.setMoneyKop(kop - amount);
                String deskripzioa =
ResourceBundle.getBundle("Etiquetas").getString("MoneyGUI.jBut
tonQuit.text");
                addMovement(email, deskripzioa, -amount);
            }
            db.getTransaction().commit();
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
        }
    }
```

2. Errefaktorizatutako kodea:

```
  public void addMoney(String email, float amount) {
      String desc = ResourceBundle.getBundle("Etiquetas").getString("MoneyGUI.jButtonAdd.text");
      updateMoney(email, amount, true, desc);
  }

  public void quitMoney(String email, float amount) {
      String desc = ResourceBundle.getBundle("Etiquetas").getString("MoneyGUI.jButtonQuit.text");
      updateMoney(email, amount, false, desc);
  }

private void updateMoney(String email, float amount, boolean isAdd, String fullDescription) {

      boolean isNewTransaction = !db.getTransaction().isActive();
      if (isNewTransaction) {
          db.getTransaction().begin();
      }

      try {
          Registered u = db.find(Registered.class, email);
          if (u != null) {
              float finalAmount = isAdd ? amount : -amount;
              u.setMoneyKop(u.getMoneyKop() + finalAmount);
              addMovement(email, fullDescription, finalAmount);
          }

          if (isNewTransaction) {
              db.getTransaction().commit();
          }
      } catch (Exception e) {
          if (isNewTransaction && db.getTransaction().isActive()) {
              db.getTransaction().rollback();
          }
          throw e;
      }
  }
```

3. Deskribapena: addMoney eta quitMoney metodoek ia gauza bera egiten zuten, kantitatearen zeinua soilik aldatuz. Transakzioen logika guztia updateMoney metodo pribatura atera da, JPAren kode bikoiztua ezabatuz.

4. Egilea: Markel Sanchez

3. Write short units of code (Maximo 15 lerro)

1. Hasierako kodea:

```
     public void initializeDB(){
          db.getTransaction().begin();
          try {
               Registered seller1=new
Registered("seller1@gmail.com","Aitor Fernandez","aurrera", 0);
               Registered seller2=new
Registered("seller2@gmail.com","Ane Gaztañaga","aurrera", 0);
               Registered seller3=new
Registered("seller3@gmail.com","Test Seller","aurrera", 0);
               Admin admin= new Admin("admin@ehu.eus", "Imanol
Egaña","aurrera");

               Date today = UtilDate.trim(new Date());
               seller1.addSale("futbol baloia", "oso polita,
gutxi erabilita", 2, 10,  today, null, false);
               seller1.addSale("salomon mendiko botak", "44
zenbakia, 3 ateraldi",2,  20,  today, null, false);
               seller1.addSale("samsung 42\" telebista",
"berria, erabili gabe", 1, 175,  today, null, false);

               seller2.addSale("imac 27", "7 urte, dena ondo
dabil", 1, 200,today, null, false);
               seller2.addSale("iphone 17", "oso gutxi
erabilita", 2, 400, today, null, false);
               seller2.addSale("orbea mendiko bizikleta",
"29\" 10 urte, mantenua behar du", 3,225, today, null, false);
               seller2.addSale("polar kilor erlojua", "Vantage
M, ondo dago", 3, 30, today, null, false);

               seller3.addSale("sukaldeko mahaia", "1.8*0.8, 4
aulkiekin. Prezio finkoa", 3,45, today, null, false);

               db.persist(seller1);
               db.persist(seller2);
               db.persist(seller3);
               db.persist(admin);

               db.getTransaction().commit();
```

```
                System.out.println("Db initialized");
            }
            catch (Exception e){
                e.printStackTrace();
            }
        }
```

2. Errefaktorizatutako kodea:

```
   public void initializeDB(){
       db.getTransaction().begin();
       try {
       createMockUsersAndSales();
       db.getTransaction().commit();
       System.out.println("Db initialized");
       }
       catch (Exception e){ e.printStackTrace(); }
       }

        private void createMockUsersAndSales() {
        Registered seller1 = new Registered("seller1@gmail.com","Aitor
Fernandez","aurrera", 0);
        Registered seller2 = new Registered("seller2@gmail.com","Ane
Gaztañaga","aurrera", 0);
        Registered seller3 = new Registered("seller3@gmail.com","Test
Seller","aurrera", 0);

        Admin admin = new Admin("admin@ehu.eus", "Imanol Egaña","aurrera");
        Date today = UtilDate.trim(new Date());

        seller1.addSale("futbol baloia", "oso polita, gutxi erabilita", 2,
10, today, null, false);
        seller1.addSale("samsung 42\" telebista", "berria, erabili gabe", 1,
175, today, null, false);
        seller2.addSale("imac 27", "7 urte, dena ondo dabil", 1, 200,today,
null, false);
        seller3.addSale("sukaldeko mahaia", "1.8*0.8, 4 aulkiekin. Prezio
finkoa", 3,45, today, null, false);

        db.persist(seller1);
        db.persist(seller2);
        db.persist(seller3);
        db.persist(admin);
        }
```

3. Deskribapena: initializeDB metodoak 30 lerro baino gehiago zituen. Objektuen eta erabiltzaileen sorrera createMockUsersAndSales() metodora atera da, metodo nagusia 9 lerrotan utziz.

4. Egilea: Markel Sanchez

4. Write simple units of code (Maximo 4 branch points)

1. Hasierako kodea:

```
   public String buySale(Sale sale, String buyerEmail)  {
        if (!db.getTransaction().isActive()) {
            db.getTransaction().begin();
        }
        try {
            Registered buyer = db.find(Registered.class,
buyerEmail);
            Sale s = db.find(Sale.class,
sale.getSaleNumber());
            Registered seller = s.getUser();

            if (buyer != null && seller != null) {
                float price = s.getPrice();

                if
(buyer.getEmail().equals(seller.getEmail())) {
                    return "Can't buy own sale";
                }
                if (buyer.getMoneyKop() < price) {
                    return "Not enough money";
                }
                buyer.setMoneyKop(buyer.getMoneyKop() -
price);
                seller.setMoneyKop(seller.getMoneyKop() +
price);

                buyer.addPurchasedSale(s);
                s.setSold(true);

                if (buyer.getFavorites() != null) {
                    buyer.getFavorites().remove(s);
                }

                addMovement(buyerEmail,
ResourceBundle.getBundle("Etiquetas").getString("Buy") + ": "
+ s.getTitle(), -s.getPrice());
                addMovement(seller.getEmail(),
ResourceBundle.getBundle("Etiquetas").getString("Sell") + ": "
+ s.getTitle(), s.getPrice());
            }
```

```
            db.getTransaction().commit();
            return null;
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
            return "Error: " + e.getMessage();
        }
    }
```

2. Errefaktorizatutako kodea:

```
  public String buySale(Sale sale, String buyerEmail) {
      if (!db.getTransaction().isActive()) {
          db.getTransaction().begin();
      }
      try {
          Registered buyer = db.find(Registered.class, buyerEmail);
          Sale s = db.find(Sale.class, sale.getSaleNumber());
          String validationError = validatePurchase(buyer, s);
          if (validationError != null) {
              return validationError;
          }

          processPurchaseTransaction(buyer, s.getUser(), s);
          db.getTransaction().commit();
          return null;
      } catch (Exception e) {
          if (db.getTransaction().isActive()) {
              db.getTransaction().rollback();
          }
          return "Error: " + e.getMessage();
      }
  }
  private String validatePurchase(Registered buyer, Sale s) {
      if (buyer == null || s.getUser() == null) {
          return "Users not found";
      }
      if (buyer.getEmail().equals(s.getUser().getEmail())) {
          return "Can't buy own sale";
      }
      if (buyer.getMoneyKop() < s.getPrice()) {
          return "Not enough money";
      }
      return null;
  }
```

```
private void processPurchaseTransaction(Registered buyer, Registered seller,
Sale s) {

       String buyDesc =
ResourceBundle.getBundle("Etiquetas").getString("Buy") + ": " +
s.getTitle();
       String sellDesc =
ResourceBundle.getBundle("Etiquetas").getString("Sell") + ": " +
s.getTitle();


       updateMoney(buyer.getEmail(), s.getPrice(), false, buyDesc);
       updateMoney(seller.getEmail(), s.getPrice(), true, sellDesc);


       buyer.addPurchasedSale(s);
       s.setSold(true);

       if (buyer.getFavorites() != null) {
           buyer.getFavorites().remove(s);
       }
   }
```

3. Deskribapena: buySale metodoak habiatutako baldintza gehiegi zituen (konplexutasun ziklomatiko handia). Baldintzak validatePurchase metodora atera dira ("Guard Clauses" erabiliz) eta transakzioaren logika processPurchaseTransaction metodora, metodo nagusiaren adarrak murriztuz.

4. Egilea: Markel Sanchez